

Lab Experiments 1 to 10

Experiment – 1: Write a Pandas program to select the distinct department IDs

Aim

To write a Pandas program to display the distinct department IDs from the DataFrame.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing department IDs.
3. Select the DEPARTMENT_ID column.
4. Use the unique() function to find distinct department IDs.
5. Display the result.

Code

```
import pandas as pd

data = {
    "DEPARTMENT_ID":
[10,20,30,40,50,60,70,80,90,100,110,120,130,140,150,160,170,180,190,200,210,220,230,240,250,260,270]
}

df = pd.DataFrame(data)

print("Distinct Department IDs:")

print(df["DEPARTMENT_ID"].unique())
```

Output

```
... Distinct Department IDs:
[ 10  20  30  40  50  60  70  80  90 100 110 120 130 140 150 160 170 180
 190 200 210 220 230 240 250 260 270]
```

Result

The program successfully displayed all the distinct department IDs from the DataFrame.

Experiment – 2: Write a Pandas program to display the IDs of employees who have done two or more jobs in the past

Aim

To write a Pandas program to identify employees who have worked in two or more jobs.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing employee and job details.
3. Group the data by EMPLOYEE_ID.
4. Count the number of jobs for each employee.
5. Display employee IDs whose job count is greater than or equal to two.

Code

```
import pandas as pd

data = {
    "EMPLOYEE_ID":[102,101,101,201,114,122,200,176,176,200],
    "JOB_ID":["IT_PROG","AC_ACCOUNT","AC_MGR","MK_REP","ST_CLERK",
             "ST_CLERK","AD_ASST","SA_REP","SA_MAN","AC_ACCOUNT"]
}

df = pd.DataFrame(data)

result = df.groupby("EMPLOYEE_ID").size()

print(result[result >= 2])
```

Output

```

... EMPLOYEE_ID
101      2
176      2
200      2
dtype: int64

```

Result

The program successfully displayed the employee IDs of employees who have worked in two or more jobs.

Experiment – 3: Write a Pandas program to display the details of jobs in descending sequence on job title

Aim

To write a Pandas program to sort job details in descending order of job title.

Algorithm

1. Import the Pandas library.
2. Create a DataFrame containing job details.
3. Sort the DataFrame using the JOB_TITLE column in descending order.
4. Display the sorted DataFrame.

Code

```

import pandas as pd

data = {
    "JOB_ID":["AD_PREP", "AD_VP", "AD_ASST", "FI_MGR", "FI_ACCOUNT"],
    "JOB_TITLE":["President",
                "Administration Vice President",
                "Administration Assistant",
                "Finance Manager",

```

```

        "Accountant"]

}

df = pd.DataFrame(data)

result = df.sort_values(by="JOB_TITLE", ascending=False)

print(result)

```

Output

```

...      JOB_ID      JOB_TITLE
0      AD_PRES      President
3      FI_MGR      Finance Manager
1      AD_VP      Administration Vice President
2      AD_ASST      Administration Assistant
4      FI_ACCOUNT      Accountant

```

Result

The program successfully sorted and displayed the job details in descending order based on the job title.

Experiment – 4: Write a Pandas program to create a line plot of the historical stock prices of Alphabet Inc.

Aim

To create a line plot showing the historical stock prices of Alphabet Inc. between two specified dates.

Algorithm

1. Import Pandas and Matplotlib.
2. Read the stock data from the CSV file.
3. Convert the Date column into datetime format.
4. Filter the records between the specified dates.
5. Plot the closing stock prices using a line chart.

6. Display the graph.

Code

```
import pandas as pd

import matplotlib.pyplot as plt

df = pd.read_csv("alphabet_stock_data.csv")

df["Date"] = pd.to_datetime(df["Date"])

data = df[(df["Date"] >= "2020-01-01") &
          (df["Date"] <= "2020-12-31")]

plt.figure(figsize=(10,5))

plt.plot(data["Date"], data["Close"])

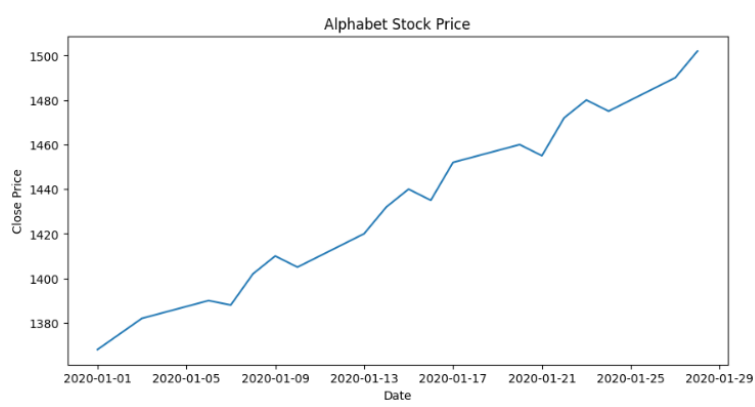
plt.title("Alphabet Stock Price")

plt.xlabel("Date")

plt.ylabel("Close Price")

plt.show()
```

Output



Result

The program successfully created a line plot showing the historical closing stock prices of Alphabet Inc. between the specified dates.

Experiment – 5: Write a Pandas program to create a bar plot of the trading volume

Aim

To create a bar plot representing the trading volume of Alphabet Inc. stock.

Algorithm

1. Import Pandas and Matplotlib.
2. Read the stock data.
3. Convert the Date column into datetime format.
4. Filter the required dates.
5. Plot the trading volume using a bar chart.
6. Display the graph.

Code

```
import pandas as pd

import matplotlib.pyplot as plt

df = pd.read_csv("alphabet_stock_data.csv")

df["Date"] = pd.to_datetime(df["Date"])

data = df[(df["Date"] >= "2020-01-01") &
          (df["Date"] <= "2020-12-31")]

plt.figure(figsize=(10,5))

plt.bar(data["Date"], data["Volume"])

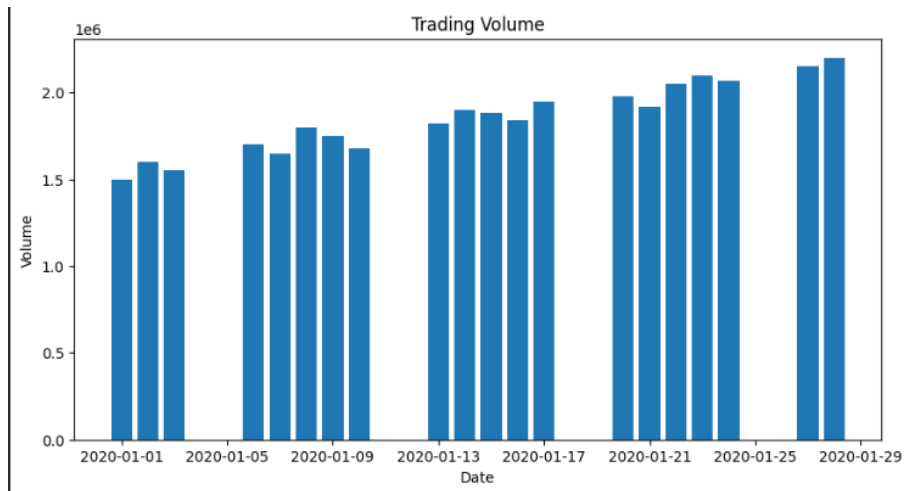
plt.title("Trading Volume")

plt.xlabel("Date")

plt.ylabel("Volume")

plt.show()
```

Output



Result

The program successfully created a bar chart representing the trading volume of Alphabet Inc. stock between the specified dates.

Experiment – 6: Write a Pandas program to create a scatter plot of trading volume and stock prices

Aim

To create a scatter plot showing the relationship between trading volume and stock prices.

Algorithm

1. Import Pandas and Matplotlib.
2. Read the stock data.
3. Convert the Date column into datetime format.
4. Filter the required dates.
5. Plot trading volume against closing stock prices using a scatter plot.
6. Display the graph.

Code

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

df = pd.read_csv("alphabet_stock_data.csv")

df["Date"] = pd.to_datetime(df["Date"])

data = df[(df["Date"] >= "2020-01-01") &
          (df["Date"] <= "2020-12-31")]

plt.scatter(data["Volume"], data["Close"])

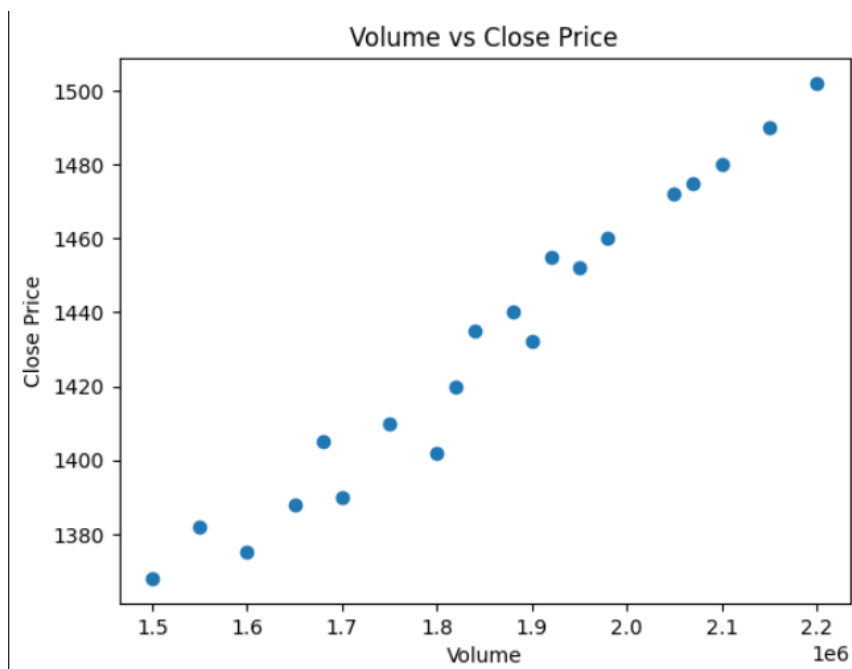
plt.title("Volume vs Close Price")

plt.xlabel("Volume")

plt.ylabel("Close Price")

plt.show()
```

Output



Result

The program successfully created a scatter plot showing the relationship between the trading volume and stock prices of Alphabet Inc.

Experiment – 7: Create a Pivot Table to find the maximum and minimum sale value

Aim

To create a Pivot Table and find the maximum and minimum sale amount for each item.

Algorithm

1. Import the Pandas library.
2. Read the sales data.
3. Create a Pivot Table using the Item column as the index.
4. Apply max and min aggregation functions to Sale_amt.
5. Display the Pivot Table.

Code

```
import pandas as pd

sales = pd.read_csv("sales_data.csv")

pivot = pd.pivot_table(
    sales,
    values="Sale_amt",
    index="Item",
    aggfunc=["max","min"]
)

print(pivot)
```

Output

	max	min
Item	Sale_amt	Sale_amt
Cell Phone	14400.0	6075.0
Desk	250.0	250.0
Home Theater	40500.0	14000.0
Television	113810.0	38336.0
Video Games	936.0	936.0

Result

The program successfully created a Pivot Table displaying the maximum and minimum sale amount for each item.

Experiment – 8: Create a Pivot Table to find the item-wise units sold

Aim

To create a Pivot Table showing the total units sold for each item.

Algorithm

1. Import the Pandas library.
2. Read the sales data.
3. Create a Pivot Table using the Item column as the index.
4. Sum the values in the Units column.
5. Display the Pivot Table.

Code

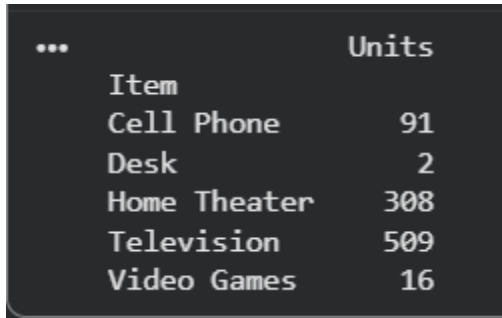
```
import pandas as pd

sales = pd.read_csv("sales_data.csv")

pivot = pd.pivot_table(
    sales,
```

```
values="Units",  
index="Item",  
aggfunc="sum"  
)  
print(pivot)
```

Output



Item	Units
Cell Phone	91
Desk	2
Home Theater	308
Television	509
Video Games	16

Result

The program successfully created a Pivot Table showing the total units sold for each item.

Experiment – 9: Create a Pivot Table to find the total sale amount region-wise, manager-wise and salesman-wise

Aim

To create a Pivot Table showing the total sale amount grouped by region, manager, and salesman.

Algorithm

1. Import the Pandas library.
2. Read the sales data.
3. Create a Pivot Table with Region, Manager, and SalesMan as the index.
4. Sum the Sale_amt values.
5. Display the Pivot Table.

Code

```
import pandas as pd

sales = pd.read_csv("sales_data.csv")

pivot = pd.pivot_table(

    sales,

    values="Sale_amt",

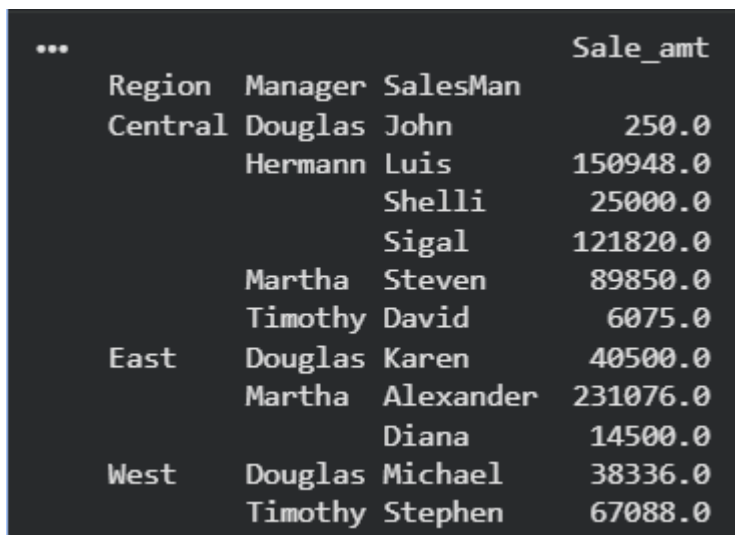
    index=["Region","Manager","SalesMan"],

    aggfunc="sum"

)

print(pivot)
```

Output



...	Region	Manager	SalesMan	Sale_amt
	Central	Douglas	John	250.0
			Hermann	150948.0
			Shelli	25000.0
			Sigal	121820.0
		Martha	Steven	89850.0
		Timothy	David	6075.0
	East	Douglas	Karen	40500.0
		Martha	Alexander	231076.0
			Diana	14500.0
	West	Douglas	Michael	38336.0
			Timothy	67088.0

Result

The program successfully created a Pivot Table displaying the total sale amount grouped by region, manager, and salesman.

Experiment – 10: Create a DataFrame with random values and highlight negative numbers

Aim

To create a DataFrame with random values and highlight the negative numbers.

Algorithm

1. Import the Pandas and NumPy libraries.
2. Generate a DataFrame with 10 rows and 4 columns of random values.
3. Define a function to highlight negative values.
4. Apply the highlighting function using DataFrame styling.
5. Display the styled DataFrame.

Code

```
import pandas as pd
import numpy as np
np.random.seed(10)
df = pd.DataFrame(
    np.random.randn(10, 4),
    columns=["A", "B", "C", "D"]
)
def highlight_negative(value):
    if value < 0:
        return "background-color: yellow"
    return ""
styled_df = df.style.map(highlight_negative)
print(df)
styled_df
```

Output

	A	B	C	D
0	1.331587	0.715279	-1.545400	-0.008384
1	0.621336	-0.720086	0.265512	0.108549
2	0.004291	-0.174600	0.433026	1.203037
3	-0.965066	1.028274	0.228630	0.445138
4	-1.136602	0.135137	1.484537	-1.079805
5	-1.977728	-1.743372	0.266070	2.384967
6	1.123691	1.672622	0.099149	1.397996
7	-0.271248	0.613204	-0.267317	-0.549309
8	0.132708	-0.476142	1.308473	0.195013
9	0.400210	-0.337632	1.256472	-0.731970

	A	B	C	D
0	1.331587	0.715279	-1.545400	-0.008384
1	0.621336	-0.720086	0.265512	0.108549
2	0.004291	-0.174600	0.433026	1.203037
3	-0.965066	1.028274	0.228630	0.445138
4	-1.136602	0.135137	1.484537	-1.079805
5	-1.977728	-1.743372	0.266070	2.384967
6	1.123691	1.672622	0.099149	1.397996
7	-0.271248	0.613204	-0.267317	-0.549309
8	0.132708	-0.476142	1.308473	0.195013
9	0.400210	-0.337632	1.256472	-0.731970

Result

The program successfully created a DataFrame with 10 rows and 4 columns of random values and highlighted all the negative numbers for easy identification.